

give basic examples of lambda function of map and forEach

Sure. In Java, lambdas are commonly used with `forEach()` and `map()` when working with collections and streams.

1. Basic `forEach()` example

`forEach()` performs an action on every element.

Java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {

        List<String> names = List.of("Ali", "Ahmed", "Sara");

        names.forEach(name -> System.out.println(name));
    }
}
```

Output:

```
Ali
Ahmed
Sara
```

The lambda is:

Java

```
name -> System.out.println(name)
```

You can read it as:

For each `name`, print the `name`.

A shorter method-reference version is:

Java

```
names.forEach(System.out::println);
```

2. `forEach()` with numbers

Java

```
List<Integer> numbers = List.of(10, 20, 30, 40);

numbers.forEach(number -> {
    System.out.println(number);
});
```

You can also perform calculations:

Java

```
numbers.forEach(number ->
    System.out.println(number * 2)
);
```

Output:

```
20
40
60
80
```

3. Basic `map()` example

`map()` is used to **transform each element into another value**.

Java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {

        List<Integer> numbers = List.of(1, 2, 3, 4, 5);

        numbers.stream()
            .map(number -> number * 2)
            .forEach(number -> System.out.println(number));
    }
}
```

Output:

2
4
6
8
10

Here:

Java

```
.map(number -> number * 2)
```

means:

1 → 2
2 → 4
3 → 6
4 → 8
5 → 10

4. `map()` with Strings

Suppose we want to convert names to uppercase:

Java

```
List<String> names = List.of("ali", "ahmed", "sara");  
  
names.stream()  
    .map(name -> name.toUpperCase())  
    .forEach(name -> System.out.println(name));
```

Output:

ALI
AHMED
SARA

It can also be written using method references:

Java

```
names.stream()  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

5. Store the result of `map()`

`map()` doesn't normally change the original list. You can create a new list:

Java

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);

List<Integer> doubled =
    numbers.stream()
        .map(n -> n * 2)
        .toList();

System.out.println(doubled);
```

Output:

```
[2, 4, 6, 8, 10]
```

The original remains:

Java

```
System.out.println(numbers);
```

```
[1, 2, 3, 4, 5]
```

Main difference

Java

```
forEach(x -> ...)
```

is normally used to **do something** with every element, such as printing:

Java

```
names.forEach(name -> System.out.println(name));
```

While:

Java

```
map(x -> ...)
```

is used to **convert/transform** every element:

Java

```
names.stream()
    .map(name -> name.toUpperCase())
    .toList();
```

A useful way to remember it is:

```
forEach → perform an action
map      → transform a value
```

For your `Course` and `Grade` example, `map()` can next be used to convert **marks** → **grade points**, and then calculate CGPA.

ChatGPT can make mistakes. Check important info.